

Interface-Scoped Dispatch: Object Composition with $O(1)$ Self-Calls

R. Scott McKinney
scott@manifold.systems

Abstract

Object composition forces a trade under static typing: independent components, or correct self-call dispatch. Forwarding-based tools supply independent components, but a component’s self-calls resolve within the component, not the composite. Trait and mixin composition preserves self-call dispatch at the expense of independent components. Neither delivers both. Achieving both at the cost of ordinary dispatch, in a statically-typed language, is the open problem.

We present *Parts*, realized as a compiler plugin for Java 8–26. The enabling mechanism is *interface-scoped dispatch* (ISD): composite identity is assigned per interface rather than per object, so self-calls route through the correct composite in arbitrary DAG compositions. The dispatch cost is one array-indexed load plus standard `invokeinterface`, structurally equivalent to vtable dispatch and empirically corroborated. *Parts* supports DAG composition with multiple composite roots per component, resolves diamond ambiguity statically, and introduces composite-scoped encapsulation.

Contents

1	Introduction	2
2	Overview	3
3	The Model	4
4	Implementation	8
4.1	Component Transformation	8
4.2	Rewriting	8
4.3	Abstract Component Support	9
4.4	Default Methods	9
4.5	@internal Enforcement	10
4.6	Safety Guarantees	10
5	Evaluation	11
5.1	Dispatch Performance	11

5.2	Example: Teaching Assistant	12
5.3	Case Study: Stream Decoration	14
5.4	Test Coverage	16
5.5	Limitations	16
6	Discussion	16
6.1	Implementation Identity and Dispatch Identity	16
6.2	Language Design Implications	17
6.3	Concurrency	18
6.4	Future Work	18
7	Related Work	18
8	Conclusion	20

1 Introduction

Object composition is the standard prescription for avoiding implementation inheritance [1]. Two properties make it a genuine replacement: *independent components*—unrestricted late-bound objects fixed at construction—and correct self-call dispatch through the composite. Static typing is what makes the two hard to combine: dynamically-typed object models have had both for decades, yet no statically-typed language provides them together at the cost of ordinary dispatch.

Forwarding-based composition (Kotlin `by` [22], Lombok `@Delegate` [23]) supplies independent components but does not support open recursion: a component’s self-calls resolve within the component because the composite is never on the dispatch path. This is the Self problem [3, 8], identified in 1986.

Trait composition (Scala `with` [9], mixins [4]) inverts the trade. Self-calls dispatch through `this` in the linearized class, so composite overrides are visible. But traits are fused into the class hierarchy at compile time, not supplied as runtime objects. Components cannot be constructed independently and passed in; composition is static.

Parts provides both. `@part` marks a class as a component; `@link` marks a field through which a composite delegates one or more interfaces to that component. Components are independent objects supplied at construction time. When a composite is constructed, a compiler plugin wires composite identity into each component, so self-calls in the component dispatch through the composite. The enabling mechanism is *interface-scoped dispatch* (ISD): assigning composite identity per interface rather than per object enables correct self-call dispatch in all *Parts* compositions.

The core contributions are:

- **Parts**: a model for object composition in which components are independent runtime objects, self-calls dispatch through the composite, and interface contracts are statically

enforced. The model is defined over a standard typed object model with a central dispatch invariant (Proposition 1) and composite-scoped encapsulation (`@internal`).

- **Interface-scoped dispatch (ISD):** the mechanism that assigns composite identity per interface rather than per object, enabling correct self-call dispatch in Parts compositions, including partial delegation (where a composite claims a proper subset of a component’s interfaces) and arbitrary DAG compositions.
- **A practical implementation:** a compiler plugin for Java 8–26 realizing the model at $O(1)$ dispatch cost, with open-source release [25] and IDE integration.

2 Overview

Consider a minimal example that shows `@part` and `@link` working in a concrete single-rooted composite.

```
interface Formatter {
    String prefix();
    String format(String msg);
}

@part class LogFormatter implements Formatter {
    public String prefix() { return "INFO"; }
    public String format(String msg) { return "[" + prefix() + " "
        + msg; }
}

class AppFormatter implements Formatter {
    @link Formatter fmt;
    AppFormatter(Formatter f) { this.fmt = f; }
    @Override public String prefix() { return "APP"; }
}

new AppFormatter(new LogFormatter()).format("started"); // "[APP]
started"
```

When `AppFormatter` is constructed, the compiler wires `LogFormatter`’s composite identity for `Formatter` to `AppFormatter`. A call to `format("started")` dispatches to `LogFormatter.format()`, and the self-call `prefix()` within `format()` dispatches back through `AppFormatter`, invoking `AppFormatter.prefix()`, not `LogFormatter.prefix()`.

Because the `@link` field accepts any `Formatter` at construction time, the component is a real runtime object, not fused into the class hierarchy. A different `Formatter` implementation can be substituted without changing `AppFormatter`.

Interface contracts are enforced at compile time: the `@link` field has type `Formatter`, and the compiler verifies that `AppFormatter` implements the delegated interface. Component

identity may not escape the composite dispatch path: passing or assigning `this` as the concrete `@part` type is a compile error; interface types are required.

This example is single-rooted: `AppFormatter` is the sole composite for `LogFormatter`, and one composite identity suffices. The following section exposes the limit of the single-identity model and the per-interface generalization that resolves it.

3 The Model

Motivating example

The following example shows a component with multiple interface roots, each requiring a distinct composite identity. `Track` implements both `Streamable` and `Classifiable`; in the composed graph, `Streamable` is rooted at `Playlist` and `Classifiable` is rooted at `Library`. From the same `Track` instance, a self-call to `codec()` must route through `Playlist` and a self-call to `genre()` through `Library`. A single identity slot cannot hold both. A per-interface identity array—`$selves[IDX_I]`, at the compile-time index `IDX_I` assigned to interface `I`, holds the composite identity for `I` independently of every other interface—provides exactly one root per interface. Figures 1 and 2 show the structure.

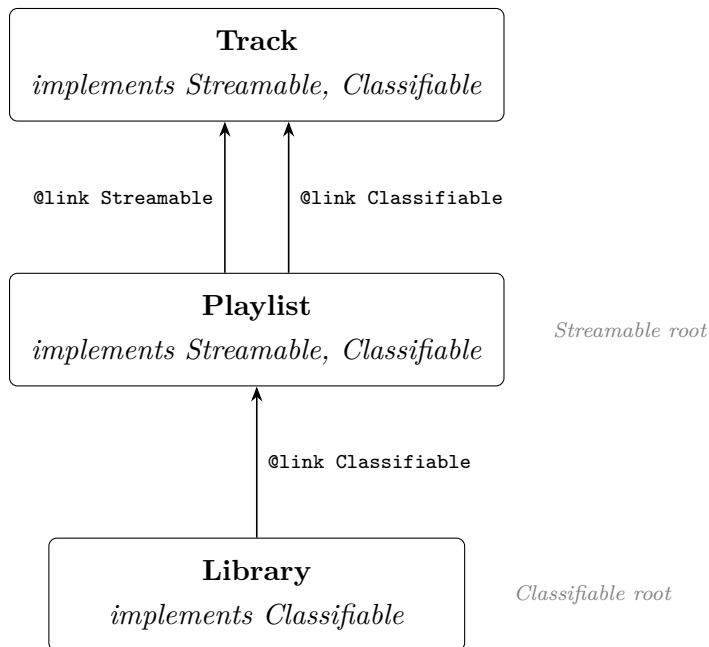


Figure 1: DAG composition structure. Arrows show `@link` fields labeled by the delegated interface. `Playlist` is the root for `Streamable` dispatch; `Library` is the root for `Classifiable` dispatch.

```

interface Streamable {
    String codec();
    void stream(String audio);
}

interface Classifiable {
    String genre();
    String classify(String title);
}

@part class Track implements Streamable, Classifiable {
    public String codec() { return "wav"; }
    public String genre() { return "unknown"; }
    public void stream(String audio) { System.out.println("[ " +
        codec() + " ] " + audio); }
    public String classify(String title) {
        stream("indexing: " + title);    // Streamable dispatch
        return genre() + ":" + title;    // Classifiable dispatch
    }
}

@part class Playlist implements Streamable, Classifiable {
    @link Streamable streamable;
    @link Classifiable classifiable;
    Playlist(Track t) { this.streamable = t; this.classifiable = t;
    }
    @Override public String codec() { return "mp3"; }
}

class Library implements Classifiable {
    @link Classifiable classifiable;
    Library(Classifiable c) { this.classifiable = c; }
    @Override public String genre() { return "jazz"; }
}

Playlist playlist = new Playlist(new Track());
Library library = new Library(playlist);
String result = library.classify("Autumn Leaves");
out.println(result);
// Streamable dispatch rooted at Playlist -> codec() -> "mp3"
// Classifiable dispatch rooted at Library -> genre() -> "jazz"
// prints: "[mp3] indexing: Autumn Leaves"
// returns: "jazz:Autumn Leaves"

```

Figure 2: DAG composition with multiple composite roots per component. `Track` carries distinct composite identities for `Streamable` (rooted at `Playlist`) and `Classifiable` (rooted at `Library`).

Definitions

We define Parts over a standard typed object model where classes implement interfaces and method dispatch is defined by the receiver’s runtime type.

Definition 1 (Component). *A component is a class annotated **@part**. Components implement one or more interfaces. A component may itself contain **@link** fields.*

Definition 2 (Link). *A link is a field annotated with **@link**. Its declared type is an interface or an abstract **@part** class. A link claims the interfaces common to its field type’s interface hierarchy and the enclosing composite’s. The plugin enforces **private** and **final** on link fields; explicit modifiers are redundant.*

Definition 3 (Composite). *A composite is a class containing one or more **@link** fields. A composite implements all interfaces claimed by its links.*

Definition 4 (Delegation surface). *The delegation surface of a **@part** is its interface hierarchy, excluding any interfaces marked **@internal** in its **implements** clause.*

Definition 5 (Composite identity). *When a component is wired to a composite, it receives a composite identity: a reference to the composite object. Self-calls within the component (calls to methods in the component’s interfaces) dispatch through the composite identity rather than **this**.*

Definition 6 (Interface-scoped dispatch). *Interface-scoped dispatch is the dispatch mechanism by which the composition receiver for a self-call is determined by the interface of the called method, not by the object identity of the caller. Each interface in a component’s delegation surface is assigned an independent composite identity; self-calls route through the composite for that interface.*

Definition 7 (Transitive propagation). *A **@part** may itself contain **@link** fields. When such a part is wired to an outer composite, composite identity propagates transitively: the outer composite becomes the dispatch root for each interface it claims throughout the delegation chain. Interfaces not claimed by the outer composite retain the dispatch roots established by inner composites.*

Definition 8 (Diamond resolution). *When two links in a composite share a common interface, exactly one must declare **@link(share=T.class)** for the shared interface. That link is authoritative; the other delegates only its non-shared interfaces, and the shared interface is excluded from propagation when the non-authoritative link is wired. Unresolved diamonds are compile errors.*

Two practical extensions—composite-scoped encapsulation (**@internal**) and abstract components with deferred linkage (**asLink()**)—are defined and implemented in Section 4.

Dispatch rule. Traditional object-oriented dispatch resolves a call through the receiver’s object identity:

$$\text{call } m() \text{ on } o \longrightarrow vtable(o, m)$$

Interface-scoped dispatch replaces the single object identity with a per-interface composite identity. A self-call $m()$ in component c where $m \in I$ resolves through the composite identity for I :

$$\text{self-call } m() \in I \text{ in } c \longrightarrow \$selves_c[I].m()$$

For self-calls within `@part` methods, the dispatch triple is $(\text{component}, \text{interface}, \text{method})$ rather than the conventional $(\text{object}, \text{method})$. Figure 3 contrasts this per-interface routing with a single shared composite identity. This is *open recursion*—a method’s self-calls late-bound to the most-derived receiver—with that receiver generalized from the object itself to the composite for each interface. Forwarding is the closed case: a component’s self-calls stay in the component, and the composite never enters the recursion.

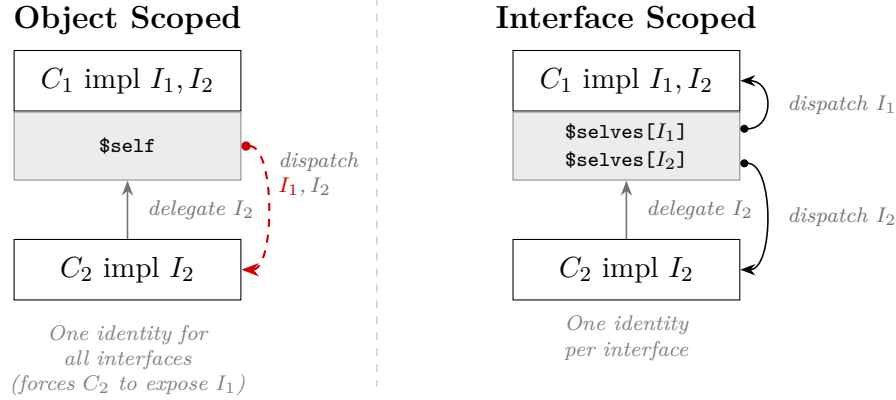


Figure 3: Object-scoped vs. interface-scoped dispatch. In both panels C_2 (impl I_2) delegates I_2 to C_1 (impl I_1, I_2). *Left*: `$self` holds the full composite identity; a self-call $m() \in I_1$ inside C_1 routes through `$self = C_2`, the outer composite, requiring C_2 to implement I_1 to avoid a cast failure, a constraint C_2 may not satisfy. *Right*: each `$selves[Ii]` slot holds a scoped identity; $m() \in I_1$ dispatches through `$selves[I1]`, routing directly to C_1 without involving C_2 ; $n() \in I_2$ dispatches through `$selves[I2]`, routing to C_2 .

Proposition 1. *For any component P in a delegation chain for interface I rooted at composite C : a self-call `this.m()` within P where $m \in I$ dispatches to C .*

The invariant is established by construction. When composite C establishes a link to component P for interface I , the plugin-generated `$linkPartToSelf` sets $\$selves_P[I] \leftarrow C$ and recurses into P ’s own links, propagating C ’s identity for each interface C claims and leaving all other slots unchanged. Each interface of a component resolves through a single `$selves` slot. Wiring an outer composite that claims I re-roots this slot to that composite and leaves the slots for interfaces it does not claim unchanged, so the slot holds exactly one root at any time and stabilizes once the outermost composite claiming I is wired; C denotes that final root. Each write occurs during the wiring composite’s construction. Link fields are `final` (enforced by the plugin); once the composition rooted at C is safely published, the Java Memory Model guarantees any thread that observes C sees the fully wired `$selves` slots, which are not written thereafter. Self-calls in P where $m \in I$ are rewritten by the plugin to `((I) $selves[IDX_I]).m()`; when m belongs to multiple interfaces (the diamond case),

I is determined at compile time by diamond resolution (Section 4). For self-calls through the delegation surface, there is no code path in a `@part` method that dispatches through anything other than the indexed slot. Because the slot holds C once the composition is constructed and is not written thereafter, the invariant holds at every point a self-call can execute. Before wiring, each slot is initialized to `this`; an unwired component dispatches on itself, consistent with standard object-oriented dispatch, and the Proposition is vacuously satisfied (no delegation chain exists). A full mechanized proof in a Featherweight Java subset is a natural direction for follow-on work (Section 6.4).

4 Implementation

The implementation is a `javac` plugin operating on the compiler’s AST during two phases. In ENTER, the plugin generates structural scaffolding: the `$selves` composite identity array and `$linkPartToSelf` wiring method for each `@part` class, delegation method skeletons for each composite class, and (for abstract components) the `$Impl` subclass and `asLink()` factories. In ANALYZE, after type resolution, self-calls in `@part` bodies are rewritten, `@link` field assignments are rewritten to invoke wiring, and `@internal` visibility is enforced. No bytecode manipulation is performed; all transformation is at the AST level and subject to standard type checking.

4.1 Component Transformation

Each `@part` class holds a `$selves` array indexed by interface ordinal, assigned at compile time:

```
private final Object[] $selves = {this, ..., this};
// after wiring: $selves[IDX_I] = composite identity for
// interface I
```

An unwired component dispatches on itself; once wired, dispatch routes through the composite identity. `$linkPartToSelf` updates each relevant slot and recurses through `@link` fields in the component whose delegation surface includes the interface being propagated, leaving all other slots unchanged, with cycle detection at each assignment.

4.2 Rewriting

Within a `@part` method body, every call `this.m(args)` where `m` is declared in interface I is rewritten to:

```
((I) $selves[IDX_I]).m(args)
```

The cast to I selects the dispatch interface explicitly; when diamond resolution has assigned m to a specific authoritative link, I is that link’s interface, determined at compile time.

A composite implements every interface claimed by its links. For each method of a claimed interface that the composite does not declare itself, the plugin generates a delegating method

that forwards through the corresponding `@link` field:

```
R m(args) { return link.m(args); }    // void methods omit the
    return
```

A method the composite declares itself is left as written, and no forwarder is generated for it. When two links share an interface, the forwarders for the shared methods route through the authoritative link (Definition 8). The forwarder is a plain forward; routing a component's self-call back to the composite is handled by that component's `$selves` slot (above), not by the forwarder.

In composite classes, each `@link` field assignment `this.link = expr` is rewritten to invoke `$linkPartToSelf`, propagating the composite identity into the component. An `instanceof` check guards the call so that non-`@part` assignments pass through unchanged.

4.3 Abstract Component Support

A `@part` class may be abstract, deferring some interface methods to the composite. Linking an abstract part obligates the composite to implement those methods or to be declared abstract, enforced at compile time: the plugin generates a delegating method for every interface method the part implements and none for a method it leaves abstract, so a concrete composite is missing exactly those methods and `javac` rejects it unless they are supplied directly. This is the compositional analog of the rule that a concrete class must implement its inherited abstract methods.

To make an abstract part instantiable for linking, the compiler generates a private `$Impl` subclass whose stubs throw `DelegationLinkageError`, and an `asLink()` factory per non-private constructor:

```
private static final class $Impl extends Foo {
    $Impl(/* Foo constructor params */) { super(/* Foo
        constructor params */); }
    @Override public T m(Params p) {
        throw new DelegationLinkageError("abstract method called
            on an unlinked part");
    }
}
public static Foo asLink(/* Foo constructor params */) { return
    new $Impl(/* same */); }
```

Once wired, the stubs are unreachable: self-calls dispatch through `$selves[IDX_I]` to the composite, whose implementation the compile-time obligation guarantees. The stubs remain only as a runtime guard for an `asLink()` instance invoked before it is wired.

4.4 Default Methods

A default method's body is declared in the interface, not in a `@part`, so the self-call rewrite of Section 4 never sees its self-calls. Invoked on a wired component, an inherited default

would run with `this` bound to the component, and its self-calls would resolve within the component rather than through the composite.

For a default method a component does not override, the plugin generates a forwarder that routes on wiring state. Unwired, it runs the interface default directly (`I.super.m()`, `this` the component); this is the standalone case. Wired, it invokes the interface’s default *implementation* with the composite as receiver, so the default body’s self-calls dispatch through the composite. A default the component overrides needs no such forwarder: the override is an ordinary `@part` method whose self-calls are rewritten through `$selves` like any other. An explicit `I.super.m()` written in component code is rewritten to the same composite-rooted invocation.

Invoking a default body on a chosen receiver has no plain-bytecode form (no instruction invokes an interface default on an object other than the current receiver), so the wired reroute compiles to an `invokedynamic` call site. Its bootstrap method resolves, once per site at linkage, a handle that invokes the interface’s default body with the composite as receiver, and returns a constant call site. The target is fixed (the interface’s default body); only the receiver varies, as a call argument. The site is therefore monomorphic, and once linked the JIT treats the handle as a constant and inlines the default body into the caller: dispatch on par with the array-indexed `invokeinterface` of Section 5. The only addition over the concrete-method path is the forwarder’s wiring-state test, a single branch on the final `$selves` slot that resolves one way once the composition is constructed. The decorator and template-method patterns of Section 5 stay $O(1)$ whether their template is written as a concrete `@part` method or as an interface default.

4.5 @internal Enforcement

`@internal` may annotate an interface method or an interface type literal in a class’s `implements` clause, with distinct effects. An interface method marked `@internal` is not accessible to code external to the composite; calls from outside the composite are compile errors, while calls via `@link` field references within the composite are permitted. An interface type literal marked `@internal` in a class’s `implements` clause removes that interface from the class’s delegation surface: no external composite may claim it via a `@link` field.

4.6 Safety Guarantees

Cycle detection. If a component is wired to itself directly or transitively, `DelegationLinkageError` is thrown at assignment time during the recursive `$linkPartToSelf()` call.

Deferred method access. The obligation that a wired composite implement an abstract part’s deferred methods is enforced at compile time; the residual case is an `asLink()` instance invoked before it is wired into any composite, where the `$Impl` stub throws `DelegationLinkageError`, the compositional analog of `AbstractMethodError`.

Self-preservation. Inside a `@part`, `this` use is guarded. Field access, private calls, and callbacks through the part’s own `@link` fields (the compositional analog of `super`) are all

permitted. Passing or assigning `this` as the concrete `@part` type is not: that is a compile error, since a caller holding the bare component could invoke methods outside the composite dispatch path and violate Proposition 1. The only `this` that can leave is therefore interface-typed, and the reference that crosses is the `$selves` slot for that interface: the component’s composite identity for it, which may be the component itself. A client receiving it reaches the composite’s overrides; recovering some other interface by reflection or downcast can still reach a lower root (Section 5.5).

5 Evaluation

5.1 Dispatch Performance

The dispatch path for a self-call through a composite is:

1. Load `$selves[IDX_I]`, an array-indexed load with compile-time constant index
2. Cast to `I`
3. `invokeinterface m`, standard JVM virtual dispatch

This is $O(1)$. The parallel to vtable dispatch is direct: a vtable call is an indexed slot load followed by an indirect call. Call sites in typical compositions are monomorphic or bimorphic, the JIT’s most aggressively optimized scenarios. Composite dispatch adds at most one additional virtual dispatch over direct delegation.

Benchmark. We verify this empirically using JMH [24] in average-time mode. Each scenario calls `compute()` through a composite root at chain depth d (1–6). The *baseline* traverses d `@link` delegation hops to a leaf that returns directly, with no self-call; the *ISD* variant traverses the same hops but the leaf issues a self-call (`$selves[IDX_I].scale()`), dispatching through the composite root. The delta between the two isolates the cost of the self-call dispatch mechanism alone, independent of delegation depth. All runs: 3 JVM forks, 5 warmup iterations, 10 measurement iterations per fork (30 data points per cell); errors are $\pm$ half-width of the 99.9% confidence interval. A separate single-dispatch calibration run (`invokeinterface` on a monomorphic call site, same harness) yields 0.46 ns/op; the delta values in Table 1 are therefore roughly 1–2 $\times$ a single virtual dispatch.

Table 1: Self-call dispatch cost by delegation depth (JMH, average-time mode; ns/op, $\pm 99.9\%$ CI half-width). The delta isolates the cost of the self-call dispatch over plain delegation. All values are rounded from unrounded measurements; the delta and its interval are computed from the unrounded statistics, so the delta may differ from the rounded baseline/ISD difference by 0.01 ns.

Depth	Baseline (ns)	ISD (ns)	Delta (ns)
1	0.65 ± 0.03	1.24 ± 0.03	0.59 ± 0.04
2	0.93 ± 0.05	1.65 ± 0.05	0.71 ± 0.07
3	1.25 ± 0.05	2.05 ± 0.07	0.81 ± 0.09
4	1.67 ± 0.02	2.43 ± 0.10	0.76 ± 0.10
5	2.12 ± 0.05	2.91 ± 0.05	0.79 ± 0.07
6	2.52 ± 0.11	3.46 ± 0.11	0.94 ± 0.16

The delta is stable across depths (0.6–0.8 ns at depths 1–5, 0.94 ns at depth 6), with overlapping confidence intervals at depths 2–5. At sub-nanosecond resolution, microbenchmark variance makes fine-grained trend analysis unreliable; the $O(1)$ argument is structural; the stable delta band corroborates it. The delta is consistent with one array-indexed load (`$selves[IDX_I]`) followed by `invokeinterface`.

5.2 Example: Teaching Assistant

The following example exercises DAG composition, diamond dispatch resolution, and self-call routing through the composite. A teaching assistant is both a student and a teacher; both roles share a common `Person` base.

```
interface Person { String getName(); String getTitle(); String
    getTitledName(); }
interface Student extends Person { String getMajor(); }
interface Teacher extends Person { String getDept(); }
interface TA extends Student, Teacher {}
```

Forwarding [22, 23]. Forwarding-based mechanisms inject the implementation as a real object at construction time; components are independent. But self-calls remain within the component: the composite is invisible. Delegating `Student` by `s` and `Teacher` by `t` creates a diamond: the compiler forces a manual override of every shared `Person` method. The deeper problem is that `getTitledName()`—the template method worth sharing—cannot be safely delegated: whichever branch receives it calls `getTitle()` on its own `this`, bypassing `TaImpl.getTitle()`.

```
class TaImpl(val s: Student, val t: Teacher) : TA,
    Student by s, Teacher by t {
    // Diamond: every shared Person method must be manually resolved
    override fun getName() = s.getName()
    override fun getTitle() = "TA"
```

```

// Without this override, getTitleName() forwards to
// s.getTitleName(),
// which calls s.getTitle() = "Student" -- self stays in the
// delegate:
// TaImpl(...).getTitleName() -> "Student Milton" (wrong)
override fun getTitleName() = "${getTitle()} ${getName()}"
// TaImpl(...).getTitleName() -> "TA Milton"
// (correct, but forced)
}

```

Every composite must re-implement `getTitleName()`. Adding a new template method to `Person` requires a matching override in every composite class; the maintenance cost grows with the number of composites. Lombok's `@Delegate` generates equivalent forwarders in Java; the self-call semantics are identical.

Parts. `PersonPart` holds the template once. Its self-call to `getTitle()` dispatches through the composite at each level of the delegation chain.

```

@part class PersonPart implements Person {
    private final String name;
    public PersonPart(String name) { this.name = name; }
    public String getName() { return name; }
    public String getTitle() { return "Person"; }
    public String getTitleName() { return getTitle() + " " +
        getName(); }
}

@part class StudentPart implements Student {
    @link Person person;
    private final String major;
    public StudentPart(Person p, String major) { this.person = p;
        this.major = major; }
    public String getTitle() { return "Student"; }
    public String getMajor() { return major; }
}

@part class TeacherPart implements Teacher {
    @link Person person;
    private final String dept;
    public TeacherPart(Person p, String dept) { this.person = p;
        this.dept = dept; }
    public String getTitle() { return "Teacher"; }
    public String getDept() { return dept; }
}

@part class TaPart implements TA {
    @link(share=Person.class) Student student;
    @link Teacher teacher;
    public TaPart(Student s) {
        this.student = s;
    }
}

```

```

    this.teacher = new TeacherPart(s, "Math");
}
public String getTitle() { return "TA"; }
}

TA ta = new TaPart(new StudentPart(new PersonPart("Milton"),
    "CS"));
ta.getTitledName(); // "TA Milton" -- getTitle() dispatches
    through TaPart

```

`getTitledName()` is defined once in `PersonPart` and overridden nowhere. Diamond resolution is handled statically: `@link(share=Person.class)` designates `student` as the authoritative delegate for `Person`, so `Person` is excluded from propagation when `teacher` is wired, leaving the identity established through `student` intact.

5.3 Case Study: Stream Decoration

A pattern common in the JDK—exemplified by `java.io.InputStream` [1]—is a bulk operation implemented as a loop over a finer one: `read(byte[], int, int)` calls the single-byte `read()` repeatedly. Decorators override `read()` to add behavior; the template is supposed to propagate it to bulk calls automatically. The `Readable` interface below captures this contract directly. Under forwarding, the propagation breaks.

A `CountingStream` that tracks bytes consumed overrides `read()` to increment a counter. With Lombok's `@Delegate`, `read(byte[], int, int)` is forwarded to the delegate, which calls `read()` on itself, not on `CountingStream`. The override is never reached for bulk calls.

Forwarding [23].

```

interface Readable {
    int read();
    int read(byte[] buf, int off, int len);
}

class BaseStream implements Readable {
    private final byte[] data;
    private int pos;
    BaseStream(byte[] data) { this.data = data; }
    public int read() { return pos < data.length ? (data[pos++] &
        0xff) : -1; }
    public int read(byte[] buf, int off, int len) {
        for (int i = 0; i < len; i++) {
            int b = read(); // self-call
            if (b == -1) return i == 0 ? -1 : i;
            buf[off + i] = (byte) b;
        }
        return len;
    }
}

```

```

class CountingStream implements Readable {
    @Delegate private final Readable delegate;
    private int count;
    CountingStream(Readable r) { this.delegate = r; }
    @Override public int read() {
        int b = delegate.read();
        if (b != -1) count++;
        return b;
    }
    public int count() { return count; }
}

CountingStream cs = new CountingStream(new BaseStream(data));
cs.read(buf, 0, 16); // -> delegate.read(byte[],int,int)
                    //      -> delegate.read() -- not cs.read()
cs.count();         // 0 -- bulk reads not counted

```

Parts. BaseStream is unchanged except for the @part annotation. The self-call in read(byte[], int, int) routes through CountingStream on every call path.

```

@part class BaseStream implements Readable { /* same as above */ }

class CountingStream implements Readable {
    @link Readable stream;
    private int count;
    CountingStream(Readable r) { this.stream = r; }
    @Override public int read() {
        int b = stream.read();
        if (b != -1) count++;
        return b;
    }
    public int count() { return count; }
}

CountingStream cs = new CountingStream(new BaseStream(data));
cs.read(buf, 0, 16); // -> BaseStream.read(byte[],int,int)
                    //      -> self-call read() ->
                        CountingStream.read()
cs.count();         // 16 -- all bytes counted

```

A new decorator—buffering, decryption, rate limiting—requires only overriding read(); BaseStream.read(byte[], int, int) routes its self-call through whichever composite is the dispatch root, without modification.

5.4 Test Coverage

The implementation is tested against a suite covering:

- forwarding vs. delegation semantics
- DAG composition at depth ≥ 3
- diamond detection and resolution
- abstract component / `asLink()` lifecycle
- `@internal` interface and method enforcement
- default method dispatch
- inheritance chains in `@part` hierarchies
- cycle detection (runtime `DelegationLinkageError`)

5.5 Limitations

- Using `@link`, `@part`, and `@internal` requires configuring the `javac` plugin in the build system (Maven, Gradle, or similar).
- Composite dispatch semantics are not enforced against runtime introspection that reaches past a reference's supplied static type. A client that reflects on an escaped reference, or downcasts it to an interface it was not handed, can invoke that interface below its composite root, bypassing the overrides above it; type-directed clients, which use the interface they were handed, are unaffected. This is by design.
- The plugin relies on internal `javac` APIs for AST rewriting.

6 Discussion

6.1 Implementation Identity and Dispatch Identity

A composition model that supports independent components must handle two structural scenarios:

- **Partial delegation:** a composite delegates a subset of a component's interfaces.
- **Multi-root:** a component's interfaces are claimed by separate composites, each requiring a distinct dispatch root from the same instance.

Multi-root requires a component to carry multiple simultaneous composite identities, one per interface. ISD resolves this by separating *implementation identity* (the component object) from *dispatch identity* (the composite per interface). Within ISD, partial delegation and multi-root are modeled the same way. Every interface has a single composition receiver: the composite that claims it, or the component itself when none does.

Multi-root arises naturally from partial delegation: when an outer composite claims a subset of a component’s interfaces, transitive propagation assigns different roots to the remaining interfaces, producing multi-root at the component level. The **Track** example in Section 3 demonstrates this directly: **Library** claims only **Classifiable** from **Playlist** (partial delegation), which by propagation gives **Track** separate roots for **Streamable** and **Classifiable** (multi-root).

6.2 Language Design Implications

Extending any statically-typed OO language with a composition model equivalent to **@link** / **@part** would satisfy both properties. The model addresses each failure mode that has kept implementation inheritance in use:

- **Fragile base class** [2]: a composite’s dependency surface is the interface contracts it explicitly claims, not a component’s internal implementation; changes to its encapsulated internals do not propagate. Partial delegation completes the isolation: a composite is unaffected by changes to interfaces it did not claim.
- **Forced IS-A semantics**: reuse does not require a type relationship; the composite IS-A the interface, not the component.
- **The Self problem**: a hazard in forwarding-based composition where self-calls remain within the component rather than routing through the composite; resolved by ISD.
- **Diamond ambiguity**: explicit **@link(share=T.class)** resolution; unresolved diamonds are compile errors, not silent merges.
- **Encapsulation breach**: a composite may implement a subset of its parts’ interfaces; those it does not implement are absent from its API by construction. **@internal** strengthens this further: it provides composite-scoped visibility, preventing designated interfaces from being delegated by any external composite.
- **Abstract behavior**: abstract components with deferred linkage (**asLink()**) preserve the template-method capability of abstract classes without coupling the hierarchy.

Underlying these is one shift. Implementation inheritance’s power is open recursion: a base method’s self-calls reach the most-derived overrides. Its fragility is the surface that recursion ranges over: every overridable method and protected member. Parts keeps the open recursion but bounds it to the declared interface; self-calls still reach the composite’s overrides, while the encapsulated internals that make base classes fragile stay off the recursion entirely.

Interface inheritance, which provides type contracts and is foundational to this model, is untouched. The same choice that keeps the delegation surface a contract—interfaces rather than implementation types—is what makes dispatch constant-time: a fixed, statically-known interface set lets each interface’s root be an array-indexed slot. The self-preservation constraint—that a **@part** class may not expose **this** as its concrete type—is not a limitation but the static guarantee that makes the dispatch invariant hold.

Implementation inheritance and Parts are two bindings of the same open recursion: imple-

mentation inheritance binds it to the class hierarchy, where a subclass IS-A its superclass and inherits its entire implementation surface, while Parts binds it to a held object and a declared interface, where the composite HAS-A its components and IS-A only their interfaces. That inheritance is open recursion, separable from subtyping, is Cook’s [5, 6], and the reading of inheritance as composition is Bracha and Cook’s [4]; the missing piece has been a realization with independent runtime components, static interface contracts, and dispatch at the cost of a virtual call.

6.3 Concurrency

The DAG structure is favorable for concurrent use. Component wiring occurs during construction; once the composition is fully constructed and its root safely published, the JMM guarantees all wired `$selves` state is visible to any thread that observes the root. The slots are not written after construction; no synchronization is required for dispatch. Components may contain mutable state, subject to the usual Java concurrency rules; the dispatch mechanism adds no additional synchronization points.

6.4 Future Work

Formal verification. A full mechanized proof of Proposition 1 in a small-step operational semantics would provide assurance beyond the construction argument above. The model maps cleanly to a subset of Featherweight Java, a natural starting point for that formalization.

Per-interface composite roots as a general primitive. The structural insight underlying this model—that dispatch identity can be assigned per interface rather than per object—opens a solution space that prior single-composite-identity models cannot reach. Problems currently addressed suboptimally through dynamic dispatch, AOP weaving, or collapsed component hierarchies may admit statically sound solutions within this primitive.

7 Related Work

Static typing is the baseline requirement; models without it are included where they complete the landscape.

Forwarding. Kotlin’s `by keyword` [22] generates forwarding methods at compile time; Lombok’s `@Delegate` [23] does the same in Java; Scala 3’s `export` keyword [21] generates forwarding methods for delegated members. In all cases, a component’s self-calls resolve within the component; the composite host is invisible. The mechanism is forwarding, not delegation. Scala provides both mechanisms in the same language: `export` for forwarding with independent components, and traits for correct self-call dispatch. Yet no combination of the two satisfies both requirements. Büchi and Weck’s generic wrappers [13] give the same forwarding choice a formal, soundness-proven treatment, and are the closest prior analysis

of the identity-escape hazard our self-preservation constraint prevents (Section 4), though they decline the disciplines we adopt as too restrictive for arbitrary legacy objects.

Scala traits / abstract type members. Odersky and Zenger [9] introduced statically-typed mixin composition with abstract type members. Scala’s `with` clause linearizes traits into the target class at compile time. Self-calls dispatch through the outermost class in the linearized hierarchy. Linearization also fuses the component into the class hierarchy: trait implementations are resolved at compile time and cannot be supplied as late-bound objects at construction. A composite that holds its own implementation of one interface while delegating another from the same component requires independent component objects, which linearization precludes.

Prototype-based delegation. Lieberman [3] introduced *open delegation*: an object may delegate unanswered messages to a prototype, with *self* remaining the original receiver throughout the chain. Self [8] realized this in a working language. These systems support correct self-call dispatch with late-bound components in arbitrary delegation graphs. Static type guarantees are absent by design.

Mixins. Bracha and Cook [4] formalized mixins as parameterized abstract subclasses. Mixins extend through linearization: the mixin is applied to a chain at class definition time, not supplied as a late-bound object at construction. Self-calls dispatch through `this` in the linearized result.

Traits. Schärli et al. [7] proposed traits as stateless, composable units of behavior. Trait composition is a compile-time operation: methods are flattened into the target class, not supplied as late-bound objects at construction. Self-calls dispatch through `this` in the composed class.

Multiple inheritance. Stroustrup [10] introduced virtual base classes and virtual inheritance in C++ to handle diamonds. The result is a single linearized object with a merged vtable; components are not preserved as independent objects.

Kniesel / Lava. Kniesel’s “type-safe delegation” [11, 12] shares this paper’s goal most closely: components are real runtime objects supplied at construction, types are statically enforced, and DARWIN binds `self` to the original receiver so self-calls route through the composite rather than staying in the component. The realized cost is not constant. DARWIN compiles to per-message parent casts and forwarding adapters; by Kniesel’s own analysis the adapters “accumulate in front of the adapted objects,” and the typical configuration (an abstract or interface parent type with a concrete subtype instance, the same case `@link` targets) “can be slowed down significantly” [12, §7.4]. The proposed remedies either double every method (“weak customization,” whose payoff Kniesel calls “questionable”) or defer to the JIT a cost elimination that “remains to be evaluated” [12, §7.5]; the system was realized only in Lava, a research language. Kniesel dismissed the static alternative of storing identity in the component on two grounds: “sharing of one parent by multiple delegating children

cannot be expressed at all,” and recursive delegation could be simulated only “with significant run-time and software maintenance costs” [11]. Interface-scoped dispatch reopens that route on both counts: indexing the stored identity by interface gives multiple composites claiming disjoint interfaces of a shared component each its own root, and transitive propagation wires recursive delegation chains at construction, so dispatch stays constant-time and the chain adds no per-call cost or maintenance burden.

Delegation Layers. Ostermann’s delegation layers [14] (ECOOP 2002) combine delegation with virtual classes to compose collaborations dynamically at runtime; self-calls route correctly within a composed collaboration. The unit composed is the collaboration (a layer of virtual nested classes bound to a common enclosing type), not the individual object; participants are members of that collaboration, not components supplied independently per interface.

Virtual classes and family polymorphism. The virtual-class lineage (gbeta [17], CaesarJ [18], Object Teams/Java [19], and J& [16]) achieves open class composition through virtual nested classes within a common enclosing family type; Ernst, Ostermann, and Cook [15] provide the definitive formal treatment. Components are runtime instances of virtual classes, but they are bound to an enclosing family type and cannot be independently supplied from outside. Self-calls dispatch correctly within the family.

Dynamic composition. Newspeak [20] treats classes as first-class late-bound arguments, enabling runtime composition of first-class module objects with correct self-call dispatch. Static type checking is absent by design.

8 Conclusion

Forwarding-based composition supplies independent components but not self-call dispatch: calls through `this` resolve within the component, not through the composite. Trait and mixin composition preserves self-call dispatch but not independent components; composition is static, linearized into the class hierarchy at compile time. No prior statically-typed approach delivers both at the cost of ordinary dispatch.

Interface-scoped dispatch resolves both. Assigning composite identity per interface rather than per object gives each self-call a well-defined route through the correct composite, at the cost of one array-indexed load plus standard `invokeinterface`, structurally equivalent to vtable dispatch. The cost holds even in the general case, where components with multiple composite roots arise by structural necessity from ordinary partial delegation.

The Self problem with independent components has gone without a practical solution since 1986. It is a large part of why object composition never displaced implementation inheritance: a self-call that reaches the composite’s overrides is the one capability that made inheritance necessary rather than merely convenient. ISD closes that gap.

References

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] L. Mikhajlov and E. Sekerinski. A study of the fragile base class problem. In *Proceedings of ECOOP*, LNCS 1445, pages 355–382, 1998.
- [3] H. Lieberman. Using prototypical objects to implement shared behavior in object-oriented systems. In *Proceedings of OOPSLA*, pages 214–223, 1986.
- [4] G. Bracha and W. Cook. Mixin-based inheritance. In *Proceedings of OOPSLA/ECOOP*, pages 303–311, 1990.
- [5] W. Cook and J. Palsberg. A denotational semantics of inheritance and its correctness. In *Proceedings of OOPSLA*, pages 433–443, 1989.
- [6] W. R. Cook, W. Hill, and P. S. Canning. Inheritance is not subtyping. In *Conference Record of POPL*, pages 125–135, 1990.
- [7] N. Schärli, S. Ducasse, O. Nierstrasz, and A. Black. Traits: Composable units of behaviour. In *Proceedings of ECOOP*, LNCS 2743, pages 248–274, 2003.
- [8] D. Ungar and R. B. Smith. Self: The power of simplicity. In *Proceedings of OOPSLA*, pages 227–242, 1987.
- [9] M. Odersky and M. Zenger. Scalable component abstractions. In *Proceedings of OOPSLA*, pages 41–57, 2005.
- [10] B. Stroustrup. Multiple inheritance for C++. In *Proceedings of the EUUG Spring Conference*, 1987.
- [11] G. Kniesel. Type-safe delegation for run-time component adaptation. In *Proceedings of ECOOP*, LNCS 1628, pages 351–366, 1999. DOI: 10.1007/3-540-48743-3_16.
- [12] G. Kniesel. *Dynamic Object-Based Inheritance with Subtyping*. PhD thesis, University of Bonn, 2000.
- [13] M. Büchi and W. Weck. Generic wrappers. In *Proceedings of ECOOP*, LNCS 1850, pages 201–225, 2000.
- [14] K. Ostermann. Dynamically composable collaborations with delegation layers. In *Proceedings of ECOOP*, LNCS 2374, pages 89–110, 2002.
- [15] E. Ernst, K. Ostermann, and W. R. Cook. A virtual class calculus. In *Conference Record of POPL*, pages 270–282, 2006.
- [16] N. Nystrom, X. Qi, and A. C. Myers. J&: Nested intersection for scalable software composition. In *Proceedings of OOPSLA*, pages 347–366, 2006.
- [17] E. Ernst. Family polymorphism. In *Proceedings of ECOOP*, LNCS 2072, pages 303–326, 2001.

- [18] I. Aracic, V. Gasiunas, M. Mezini, and K. Ostermann. An overview of CaesarJ. In *Transactions on Aspect-Oriented Software Development I*, LNCS 3880, pages 135–173, 2006.
- [19] S. Herrmann. Object Teams: Improving modularity for crosscutting collaborations. In *Proceedings of NetObjectDays*, LNCS 2591, pages 248–264, 2002.
- [20] G. Bracha, P. von der Ahé, V. Bykov, Y. Kishai, W. Maddox, and E. Miranda. Modules as objects in Newspeak. In *Proceedings of ECOOP*, LNCS 6183, pages 405–428, 2010.
- [21] EPFL. Exports — Scala 3 reference. <https://docs.scala-lang.org/scala3/reference/other-new-features/export.html>, 2024.
- [22] JetBrains. Delegation — Kotlin programming language. <https://kotlinlang.org/docs/delegation.html>, 2024.
- [23] Project Lombok. @Delegate — Lombok features. <https://projectlombok.org/features/Delegate>, 2024.
- [24] A. Shipilev and contributors. JMH: Java Microbenchmark Harness. <https://github.com/openjdk/jmh>, 2013–2024.
- [25] R. S. McKinney. manifold-delegation. <https://github.com/manifold-systems/manifold/tree/master/manifold-deps-parent/manifold-delegation>, 2026.